



Spring WebSocket

Présentation

1

Spring WebSocket

2

- ✓ Introduction
- ✓ API Côté Spring
- ✓ STOMP et Sock JS côté Serveur
- ✓ Gestion des évènements
- ✓ STOMP et Sock JS côté Client

Introduction

- Le protocole WebSocket (RFC 6455) définit la communication bidirectionnelle en duplex intégral entre le client et le serveur.
- Il s'agit d'une nouvelle fonctionnalité intéressante issue d'une longue histoire de techniques visant à rendre le Web plus interactif

Introduction

➤ L'idée générale :

- Le Serveur (réalisé dans une technologie x ou y) écoute
- Le client ouvre une connexion au serveur (généralement en JavaScript)
- Cette connexion reste ouverte tant que les deux parties sont d'accords

➤ Cette approche permet au serveur de faire du *push* d'information aux clients

- Le client réagit par callback

Introduction

- Spring 4 via son module spring-websocket prend en charge la norme de l'API Java WebSocket (JSR-356)
- Il permet l'utilisation de
 - SockJS : pour la mécanique des sockets Java Script
 - STOMP : pour le protocole d'échange sur les sockets
- Il est aussi possible de faire usage du JMS

Quand faire usage de WebSocket

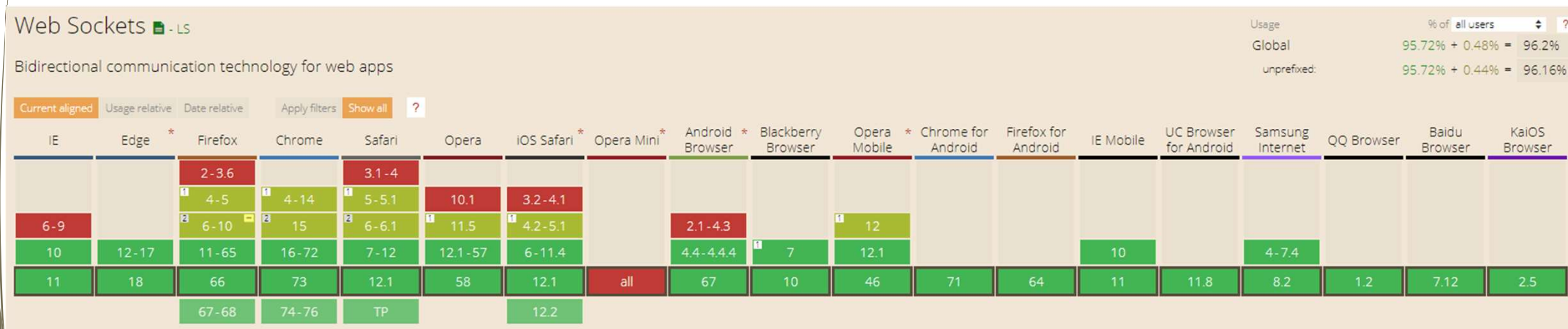
- Elles peuvent rendre une page Web dynamique et interactive.
 - les actualités,
 - les e-mails,
 - les flux sociaux,
 - la collaboration,
 - les jeux,
 - les applications financières
- Toutes les informations qui doivent être mis à jour de manière **dynamique et presque en temps réel**
 - Mais vous pouvez parfaitement le faire toutes les x minutes.

Quand faire usage de WebSocket

- C'est la **combinaison** des éléments suivants qui représente le meilleur exemple d'utilisation de WebSocket :
 - une faible latence (aller vite),
 - une fréquence élevée (souvent),
 - un volume élevé qui constitue (beaucoup)
- **Attention** : le comportement des WebSocket peut être affecté par des contraintes restrictives (navigateur, provider, infrastructure réseaux)

Compatibilité

➤ <https://caniuse.com/#feat=websockets>



➤ Compatible avec ~96% des navigateurs

Spring et WebSocket - Première approche

- En Spring il existe plusieurs approches pour réaliser du WebSocket
- Via l'interface `WebSocketHandler` qui représente une classe qui doit faire un traitement sur un message
 - Et ses enfants
 - ❑ `BinaryWebSocketHandler` : pour des messages binaires
 - ❑ `TextWebSocketHandler` : pour des messages de type texte

Spring et WebSocket - Première approche

➤ Exemple de déclaration :

```
import org.springframework.web.socket.WebSocketHandler;
import org.springframework.web.socket.WebSocketSession;
import org.springframework.web.socket.TextMessage;

@Component
public class MyHandler extends TextWebSocketHandler {

    @Override
    public void handleTextMessage(WebSocketSession session, TextMessage message) {
        // ...
    }
}
```

Spring et WebSocket - Première approche

- Pour fonctionner il faudra l'accompagner d'une classe qui implémente l'interface `WebSocketConfigurer`

```
import org.springframework.web.socket.config.annotation.EnableWebSocket;
import org.springframework.web.socket.config.annotation.WebSocketConfigurer;
import org.springframework.web.socket.config.annotation.WebSocketHandlerRegistry;

@Configuration
@EnableWebSocket
@ComponentScan("com.handlers")
public class WebSocketConfig implements WebSocketConfigurer {
    @Autowired
    private WebSocketHandler myHandler;

    @Override
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(this.myHandler, "/myHandler");
    }
}
```

- On peut envoyer des messages sur *url/myHandler*

Spring et WebSocket - SockJS

- Cette approche générique est assez lourde
- Elle donne cependant toute liberté sur comment gérer le *handshake*
- L'utilisation de SockJS simplifie notre configuration et garantira un meilleur fonctionnement
 - Permet de s'adapter aux contraintes restrictives (réseaux, proxy, providers, navigateur)

Spring et WebSocket - SockJS

- SockJS est composé de:
 - Le protocole SockJS
 - Le client JavaScript SockJS (une bibliothèque client à utiliser dans les navigateurs)
 - Implémentations de serveur SockJS (inclue dans le module spring-websocket)
 - Un client Java SockJS dans le module spring-websocket (à partir de la version 4.1).
- SockJS est conçu pour être utilisé dans les navigateurs. Il utilise diverses techniques pour prendre en charge un large éventail de versions de navigateurs (WebSocket, HTTP Streaming et HTTP Long Polling).

Spring et WebSocket - SockJS

- Le client SockJS commence par envoyer GET / info pour obtenir des informations de base du serveur.
- Après cela, il doit décider quel transport utiliser.
 - Si possible, WebSocket est utilisé.
 - Sinon, dans la plupart des navigateurs, il existe au moins une option de diffusion HTTP en continu.
 - Si ce n'est pas le cas, l'interrogation HTTP (longue) est utilisée.
- SockJS fait usage du Json pour échanger ses messages.
 - Par défaut du côté Spring, c'est Jackson qui s'en chargera

Spring et WebSocket - SockJS

➤ Exemple d'utilisation côté Java :

```
import org.springframework.web.socket.config.annotation.EnableWebSocket;
import org.springframework.web.socket.config.annotation.WebSocketConfigurer;
import org.springframework.web.socket.config.annotation.WebSocketHandlerRegistry;

@Configuration
@EnableWebSocket
@ComponentScan("com.handlers")
public class WebSocketConfig implements WebSocketConfigurer {
    @Autowired
    private WebSocketHandler myHandler;

    @Override
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(this.myHandler, "/myHandler").withSockJS();
    }
}
```

Spring et WebSocket - SockJS

- Cette approche donne la main sur plusieurs options de configuration exposées par **SockJsServiceRegistration**

```
...  
@Override  
public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {  
    SockJsServiceRegistration sjsr = registry.addHandler(this.myHandler, "/myHandler").withSockJS();  
    sjsr.setStreamBytesLimit(512 * 1024)  
        .setHttpMessageCacheSize(1000)  
        .setDisconnectDelay(30 * 1000);  
}  
}
```

- **Fixe**

- la taille maximal du flux binaire à 512KB (défaut à 128KB).
- la taille du cache des messages HTTP à 1000 (défaut à 100).
- le temps de déconnection à 30 secondes (défaut à 5s).

Spring et WebSocket - STOMP

- Streaming Text Oriented Message Protocol : format d'échange sur WebSocket
- Il est supporté par les serveurs JMS comme
 - Apache ActiveMQ,
 - HornetQ
 - RabbitMQ
- Il est aussi supporté par Spring

Stomp 

Spring et WebSocket - STOMP

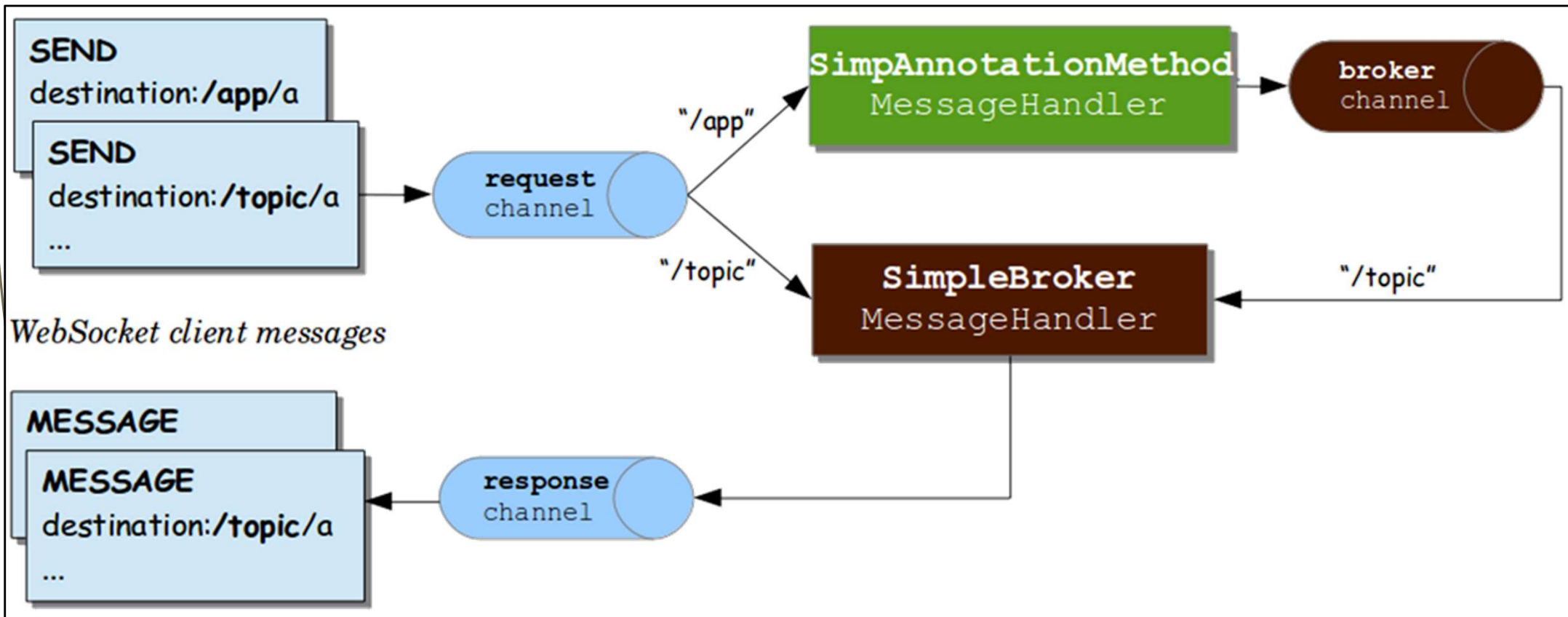
- Les clients peuvent utiliser les commandes
 - SEND : pour envoyer des messages
 - SUBSCRIBE souscrire à des topic
- Elle possède un en-tête de destination décrivant le contenu du message et les destinataires de celui-ci.
- Cela active un mécanisme simple de publication / abonnement que vous pouvez utiliser pour envoyer des messages via le broker à d'autres clients connectés ou pour envoyer des messages au serveur afin de demander l'exécution de tâches
- Spring +STOMP = Les messages sont acheminés vers les méthodes de traitement contenues dans des @Controller
- Un broker (mémoire ou autre) assure le suivi des abonnements et diffuse des messages aux utilisateurs abonnés.

Spring et WebSocket - STOMP

- L'utilisation de STOMP en tant que sous-protocole permet à Spring de fournir un modèle de programmation plus riche par rapport à l'utilisation de WebSockets bruts.
 - Pas besoin d'inventer un protocole de messagerie personnalisé et un format de message.
 - Les clients STOMP sont disponibles en Java.
 - Vous pouvez (éventuellement) utiliser des *broker* de messages (RabbitMQ, ActiveMQ, etc.) pour gérer les abonnements et les messages diffusés.
 - La logique d'application peut être organisée en un nombre illimité d'instances `@Controller` et les messages peuvent leur être acheminés en fonction de l'en-tête (par opposition à la gestion de messages WebSocket bruts avec un seul `WebSocketHandler` pour une connexion donnée).
 - Vous pouvez utiliser Spring Security pour sécuriser les messages en fonction des destinations et des types de message STOMP.

Spring et WebSocket - STOMP

20



<https://docs.spring.io/spring/docs/current/spring-framework-reference/web.html#websocket-stomp-message-flow>

Spring STOMP - Côté Java

- La mécanique est très proche sur Spring MVC
- Le contrôleur : tout comme en Spring MVC portera l'annotation **@Controller**
- Chaque méthode représentera
 - un point d'entrée
 - Et ou de sortie
 - Et ou de gestion d'erreur

Spring STOMP - Côté Java

- Chaque méthode pourra porter les annotations suivantes :
 - **@MessageMapping** : méthodes qui acheminent les messages en fonction de leur destination. Utilisable aussi sur la classe.
 - **@SubscribeMapping** : fonctionne presque comme @MessageMapping mais à n'utiliser que pour la souscription
 - **@MessageExceptionHandler** : pour gérer les erreurs. Le paramètre de la méthode sert de filtre.
 - **@SendTo** : pour renvoyer le message sur une ou plusieurs destinations
 - **@SentToUser** : pour renvoyer le message sur l'utilisateur émetteur du message d'entrée

Spring STOMP - Côté Java

23

➤ Les méthodes annotées en `@MessageMapping` pourrons avoir les paramètres suivants :

Paramètre	Description
<code>org.springframework.messaging.Message</code>	Pour accéder au message dans sa totalité
<code>org.springframework.messaging.MessageHeaders</code>	Pour n'accéder qu'aux header du message
<code>MessageHeaderAccessor</code> , <code>SimpMessageHeaderAccessor</code> , <code>StompHeaderAccessor</code>	Pour accéder aux headers de manière typée
<code>@Payload</code>	Pour accéder au message converti/traduit (par exemple en JSON) via un <code>MessageConverter</code> . Elle n'est pas obligatoire sur un objet qui ne serait d'aucun des autres types. Il est possible d'ajouter <code>@javax.validation.Valid</code> ou <code>@Validated</code> du Spring afin de gérer une validation dynamique du bean
<code>@Header</code>	Pour accéder à un élément particulier du header. Fera potentiellement usage d'un <code>org.springframework.core.convert.converter.Converter</code>
<code>@Headers</code>	Associé à une <code>java.util.Map</code> pour récupérer tout le contenu du header.
<code>@DestinationVariable</code>	Equivalent du <code>@PathVariable</code> en Spring MVC, permet d'indiquer un élément dynamique associé à l'URL du <code>@MessageMapping</code> <code>@MessageMapping("blue.{green}")</code> <code>public void handleGreen(@DestinationVariable String green) { ... }</code>
<code>java.security.Principal</code>	Représente l'utilisateur connecté lors du handshake.

Spring STOMP - Côté Java

➤ Exemple

```
@Controller
public class ChatController {
    @MessageMapping("/chat.send")
    @SendTo("/topic/public")
    public ChatMessage sendMessage(@Payload ChatMessage pMessage) {
        return pMessage;
    }
}
```

Arrive en Json et est transformé en objet par Jackson

Sera transformé en Json par Jackson

- Reçoit tous les messages à destination de **ApplicationDestinationPrefixe + "/chat.send"**
- Ne fait aucun traitement
- Renvoi le message vers tous ceux à l'écoute de **"/topic/public"**

Spring STOMP - Côté Java

25

- Envoyer un message sans être un contrôleur est tout à fait réalisable via un objet de type **SimpMessageSendingOperations** (en STOMP) ou **SimpMessagingTemplate** qui est son parent
- Il suffit de le l'injecter dans l'objet où vous en avez besoin

```
@Component
public class WebSocketEventListener {

    @Autowired
    private SimpMessageSendingOperations messagingTemplate;

    public void uneMethode() {
        ...
        this.messagingTemplate.convertAndSend("/topic/public", unMessage);
        ...
    }
}
```

Spring STOMP - Côté Java

- L'instance de `SimpMessagingTemplate` est fabriqué **automatiquement** lors de l'initialisation du broker.
- Les méthodes `convertAndSend` (et `convertAndSendToUser`) ciblent un topic ou une queue
- Le message est automatiquement transcodé pour le client (en Json par défaut en STOMP)

Spring et WebSocket - Configuration



➤ Configuration (en classe annotée)

```
@Configuration
@EnableWebSocketMessageBroker
@ComponentScan("com.mes.controleurs.websocket")
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void registerStompEndpoints(StompEndpointRegistry ser) {
        ser.addEndpoint("/ws").withSockJS();
    }

    @Override
    public void configureMessageBroker(MessageBrokerRegistry mbr) {
        mbr.setApplicationDestinationPrefixes("/app");
        mbr.enableSimpleBroker("/topic");
    }
}
```

Ne pas oublier de scanner
nos classes `@Controller`

En SockJS
Le EndPoint sera utilisé par le
code JS côté client

Le préfixe sera aussi utilisé côté
JS
Il ne sera pas visible dans le
code Java

Ici on active le SimpleBroker qui
est un broker en mémoire

Spring et WebSocket - Configuration



➤ Configuration (en XML)

```
<beans xmlns="http://www.springframework.org/schema/beans"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xmlns:context="http://www.springframework.org/schema/context"
xmlns:websocket="http://www.springframework.org/schema/websocket"
xsi:schemaLocation=
"http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context/spring-context.xsd
http://www.springframework.org/schema/websocket/spring-websocket.xsd">

<context:component-scan base-package="com.mes.controleurs.websocket" />

<websocket:message-broker application-destination-prefix="/app">
  <websocket:stomp-endpoint path="/ws">
    <websocket:sockjs />
  </websocket:stomp-endpoint>
  <websocket:simple-broker prefix="/topic" />
</websocket:message-broker>

</beans>
```

Ne pas oublier de scanner nos classes @Controller

Le préfixe sera aussi utilisé côté JS
Il ne sera pas visible dans le code Java

En SockJS
Le EndPoint sera utilisé par le code JS côté client

Ici on active le SimpleBroker qui est un broker en mémoire

Spring STOMP - Côté Java

- Les événements suivants sont disponibles :
- **BrokerAvailabilityEvent**: Indique la disponibilité ou l'indisponibilité du broker
 - **SessionConnectEvent**: Évènement publié lors de la tentative de connexion d'un client STOMP. Il est possible de récupérer via le header (`SimpMessageHeaderAccessor` ou `StompMessageHeaderAccessor`) des informations d'authentification. Réception d'un 'STOMP CONNECT'
 - **SessionConnectedEvent**: Évènement publié lors de la connexion établie d'un client STOMP. Accusé du broker d'un 'STOMP CONNECTED'.
 - **SessionSubscribeEvent**: Évènement publié lors de la réception d'un 'STOMP SUBSCRIBE'.
 - **SessionUnsubscribeEvent**: Évènement publié lors de la réception d'un 'STOMP UNSUBSCRIBE'.
 - **SessionDisconnectEvent**: Évènement publié lors de la déconnexion d'un client.

Spring STOMP - Côté Java

- Rappel sur la gestion des événements en Spring :
 - Annoter la classe en **@Component** (la placer dans un *component scan*)
 - Annoter la méthode avec **@EventListener**
 - Choisir **le paramètre** en fonction **du type** de l'évènement ciblé
 - ❑ Certain évènements peuvent supporter plusieurs paramètres

Spring STOMP - Côté Java

- Exemple, si l'on veut gérer un *SessionConnectedEvent*

```
@Component
public class WebSocketEventListener {

    @EventListener
    public void handleWebSocketConnectListener(SessionConnectedEvent event) {
        // Faire son traitement
    }

}
```

- Vous pouvez placer plusieurs méthodes dans la même classe

STOMP - Côté Java Script

- Nous pouvons faire usage de
 - <https://github.com/sockjs>
 - <https://github.com/stomp-js>
- Il suffit de les inclure dans la page web
- Au besoin, STOMP est compatible avec Angular

STOMP - Côté Java Script

➤ Ouvrir une session Stomp via Sock JS

```
var socket = new SockJS('/ws');  
stompClient = Stomp.over(socket);  
stompClient.connect(  
    {},  
    onConnected,  
    onError);
```

Le EndPoint précisé côté serveur

Information JSON du Header

Fonction de callback quand connexion ok

Fonction de callback quand connexion ko

➤ <https://stomp-js.github.io/stomp-websocket/codo/class/Client.html#connect-dynamic>

STOMP - Côté Java Script

➤ Souscrire à un topic/queue

```
stompClient.subscribe(  
  '/topic/public',  
  onMessageReceived);
```

Destination associé au SimpleBroker

Fonction de callback quand un message est reçu

```
function onMessageReceived(payload) {  
  var message = JSON.parse(payload.body);  
  // Traitement  
}
```

Le message sera en JSON par défaut

STOMP - Côté Java Script

➤ Envoyer un message

```
stompClient.send(  
    "/app/chat.add",  
    {},  
    JSON.stringify(monObjMsg));
```

Information
JSON du
Header

Destination associé au
@Controller/@RequestMapping

Le message sera en JSON par
défaut.
Attention : bien respecter le
format côté serveur

➤ <https://stomp-js.github.io/stomp-websocket/codo/class/Client.html#send-dynamic>

Travaux pratiques

➤ Exo 23 : Lets Chat in WebSocket



Pour aller plus loin

- <https://docs.spring.io/spring/docs/current/spring-framework-reference/web.html#websocket>
- <https://medium.com/oril/spring-boot-websockets-angular-5-f2f4b1c14cee>